CI/CD — 3–4 YOE Scenario-Based Q&A

1. CI/CD pipeline is failing. What will you check?
Ans: First, I check the pipeline execution and identify the exact stage where the failure occurred. I look at the job logs and error message to understand whether the issue is related to code, dependencies, build, testing, or deployment.
Then I verify the recent code changes, pipeline configuration, environment variables, and required tools. After identifying the root cause, I fix the issue and run the pipeline again to verify that all stages complete successfully.
Simple ga gurthupettuko: Pipeline fail ayithe → first ye stage lo fail ayindo chudu → job logs and error check cheyyi → recent code changes verify cheyyi → dependencies and configuration check cheyyi → issue fix chesi pipeline malli run cheyyi.
Eng: If the CI/CD pipeline fails → first identify the failed stage → check job logs and the error → verify recent code changes → check dependencies and configuration → fix the issue and run the pipeline again.
Example:
Pipeline run → ye stage failed ayyindo chudu.
Build logs → exact error ekkada vachindo check cheyyi.
git diff → recent code changes verify cheyyi.
Required configuration and dependencies correct chesi → pipeline malli run cheyyi → all stages successful ayyaya verify cheyyi.

---

2. CI pipeline build stage is failing. What will you check?
Ans: First, I check the build logs and identify the exact command that is failing. Then I verify the application dependencies, build configuration, runtime version, and required tools installed on the CI agent.
I also compare the CI environment with the local development environment to identify any differences. After fixing the dependency or configuration issue, I rerun the build and verify the generated artifact.
Simple ga gurthupettuko: Build stage fail ayithe → first failed command chudu → build logs check cheyyi → dependencies verify cheyyi → runtime and tools versions check cheyyi → CI and local environment compare cheyyi → issue fix chesi build malli run cheyyi.
Eng: If the build stage fails → first identify the failed command → check build logs → verify dependencies → check runtime and tool versions → compare CI and local environments → fix the issue and run the build again.
Example:
Build job → failed command identify cheyyi.
java -version → correct Java version unda check cheyyi.
mvn dependency:tree → required dependencies resolve avutunnaya chudu.
mvn clean package → build locally reproduce avutunda verify cheyyi.
Issue fix chesi → pipeline build malli run cheyyi.

---

3. CI/CD pipeline is failing during unit tests. What will you check?
Ans: First, I check the test output and identify which test cases are failing. Then I determine whether the failure is caused by a code change, dependency issue, test configuration, or environment difference.
I also verify the test reports and compare the failing tests with the recent code changes. I fix the actual issue instead of simply skipping the tests and rerun the pipeline.
Simple ga gurthupettuko: Unit tests fail ayithe → first ye test fail ayindo chudu → test output check cheyyi → recent code changes verify cheyyi → dependencies and test configuration check cheyyi → test report chudu → issue fix chesi tests malli run cheyyi.
Eng: If unit tests fail → first identify the failed test → check test output → verify recent code changes → check dependencies and test configuration → review the test report → fix the issue and run the tests again.
Example:
mvn test → failed test identify cheyyi.
target/surefire-reports → test failure details chudu.
git diff → recent code change check cheyyi.
Failed test root cause fix chesi → mvn test malli run cheyyi → all tests pass ayyaya verify cheyyi.

---

4. CI/CD pipeline is unable to download dependencies. What will you check?
Ans: First, I check the pipeline logs and identify which dependency download is failing. Then I verify the package repository URL, network connectivity, authentication, and dependency version.
I also check whether the CI agent has access to the required repository and whether there are proxy or firewall restrictions. After fixing the repository or network issue, I rerun the dependency installation.
Simple ga gurthupettuko: Dependencies download avvakapothe → first ye package fail ayindo chudu → repository URL check cheyyi → network connectivity verify cheyyi → authentication and permissions chudu → proxy or firewall check cheyyi → issue fix chesi dependencies malli download cheyyi.
Eng: If dependencies cannot be downloaded → first identify the failed package → check the repository URL → verify network connectivity → check authentication and permissions → verify proxy or firewall restrictions → fix the issue and download the dependencies again.
Example:
npm install → ye dependency download fail ayindo chudu.
npm config get registry → correct package registry unda verify cheyyi.
curl -I https://registry.npmjs.org → registry connectivity check cheyyi.
Authentication or network issue fix chesi → npm install malli run cheyyi.

---

5. CI/CD pipeline is taking too much time. What will you check?
Ans: First, I identify which pipeline stage is taking the most time. Then I check whether the delay is caused by dependency downloads, build time, tests, Docker image builds, or deployment operations.
I also check whether caching can be used for dependencies and build artifacts. After identifying the slow stage, I optimize that stage and verify the total pipeline execution time.

Simple ga gurthupettuko: Pipeline slow ga unte → first ye stage ekkuva time tiskuntundo chudu → dependencies, build, tests or Docker stage check cheyyi → repeated downloads unnaya verify cheyyi → caching use cheyyagalama chudu → slow stage optimize chesi pipeline time verify cheyyi.
Eng: If the pipeline is slow → first identify the stage taking the most time → check dependencies, build, tests, or Docker stages → verify repeated downloads → check whether caching can be used → optimize the slow stage and verify the pipeline time.
Example:
Pipeline logs → each stage execution time chudu.
npm install → dependencies every time download avutunnaya check cheyyi.
CI cache configuration → dependency cache available unda verify cheyyi.
Slow stage optimize chesi → pipeline malli run cheyyi → execution time compare cheyyi.

---

6. Docker image build is failing inside the CI/CD pipeline. What will you check?
Ans: First, I check the Docker build logs and identify the exact Dockerfile instruction that failed. Then I verify the base image, application files, dependencies, and build context.
I also check whether the CI agent has network access to download packages and pull the base image. After fixing the Dockerfile or environment issue, I rebuild the image and verify that it is created successfully.
Simple ga gurthupettuko: CI lo Docker build fail ayithe → first failed Dockerfile step chudu → build logs check cheyyi → base image and files verify cheyyi → dependencies and network check cheyyi → issue fix chesi Docker image malli build cheyyi.
Eng: If the Docker image build fails in CI → first identify the failed Dockerfile step → check build logs → verify the base image and files → check dependencies and network → fix the issue and build the Docker image again.
Example:
docker build -t myapp:latest . → Image build fail ayyindo chudu.
Docker build output → exact error ekkada vachindo check cheyyi.
cat package.json → required dependencies unnaya check cheyyi.
curl -I https://registry.npmjs.org → package registry connectivity unda check cheyyi.
Issue fix chesi → docker build -t myapp:latest . → image successfully build ayyinda verify cheyyi.

---

7. Docker image push is failing in the CI/CD pipeline. What will you check?
Ans: First, I verify that the Docker image has the correct registry, repository, and tag. Then I check whether the CI pipeline is authenticated with the container registry.
I also verify whether the configured credentials have permission to push to the repository and whether the CI agent can reach the registry. After correcting the authentication or permission issue, I push the image again.
Simple ga gurthupettuko: Image push fail ayithe → first image tag check cheyyi → registry login verify cheyyi → repository permission check cheyyi → registry connectivity chudu → issue fix chesi image push malli run cheyyi.
Eng: If the Docker image push fails → first check the image tag → verify registry authentication → check repository permissions → verify registry connectivity → fix the issue and push the image again.
Example:
docker images → image correct tag tho unda chudu.
docker login <registry> → registry authentication verify cheyyi.
docker push <registry>/myapp:latest → push error em vastundo chudu.
Permission or authentication issue fix chesi → docker push <registry>/myapp:latest → image push successful ayyinda verify cheyyi.

---

8. CI/CD pipeline deployment stage is failing. What will you check?
Ans: First, I check the deployment job logs and identify the exact deployment command that failed. Then I verify the target environment, credentials, deployment configuration, and required resources.
If the deployment is to Kubernetes, I check the cluster connectivity, namespace, manifests, and permissions. After fixing the deployment issue, I run the deployment again and verify the application status.
Simple ga gurthupettuko: Deployment stage fail ayithe → first failed command chudu → target environment verify cheyyi → credentials and configuration check cheyyi → Kubernetes ayithe cluster, namespace and permissions chudu → issue fix chesi deployment malli run cheyyi.
Eng: If the deployment stage fails → first identify the failed command → verify the target environment → check credentials and configuration → for Kubernetes, check the cluster, namespace, and permissions → fix the issue and deploy again.
Example:
kubectl cluster-info → CI agent cluster ki connect avutunda check cheyyi.
kubectl get ns → required namespace unda chudu.
kubectl auth can-i create deployment -n prod → deployment permission unda verify cheyyi.
Issue fix chesi → kubectl apply -f deployment.yaml -n prod → deployment successful ayyinda verify cheyyi.

---

9. CI/CD pipeline is deploying to the wrong environment. What will you check?
Ans: First, I verify the branch, pipeline variables, environment configuration, and deployment target. Then I check whether the pipeline is selecting the correct Kubernetes cluster, namespace, or server for that environment.
I also verify environment-specific configuration and approval rules. After identifying the incorrect configuration, I correct the pipeline and confirm that the deployment targets the expected environment.
Simple ga gurthupettuko: Wrong environment ki deploy ayithe → first branch and pipeline variables chudu → target environment verify cheyyi → cluster and namespace check cheyyi → environment configuration and approvals verify cheyyi → pipeline configuration fix chesi correct environment lo deploy ayyinda check cheyyi.
Eng: If the pipeline deploys to the wrong environment → first check the branch and pipeline variables → verify the target environment → check the cluster and namespace → verify environment configuration and approvals → fix the pipeline and confirm the correct deployment target.
Example:
echo $ENVIRONMENT → pipeline ye environment select chesindo chudu.

kubectl config current-context → correct cluster ki connect ayyinda check cheyyi.
kubectl get ns → correct namespace identify cheyyi.
Pipeline variables correct chesi → deployment malli run cheyyi → expected environment lo application deploy ayyinda verify cheyyi.

---

10. CI/CD pipeline secrets are exposed in the logs. What will you do?
Ans: First, I immediately identify which secret was exposed and make sure it is no longer printed by the pipeline. If the secret is sensitive, I rotate or revoke it because the old value should no longer be trusted.
Then I store the secret using the CI/CD platform's secure secret mechanism and mask it from logs. I also review the pipeline configuration and source code to prevent the same secret from being exposed again.
Simple ga gurthupettuko: Secret logs lo expose ayithe → first secret identify chesi output lo printing stop cheyyi → sensitive secret ayithe rotate or revoke cheyyi → secure secret storage use cheyyi → logs lo mask cheyyi → pipeline configuration check chesi issue repeat avvakunda chudu.
Eng: If a secret is exposed in logs → first identify the secret and stop printing it → rotate or revoke the sensitive secret → use secure secret storage → mask it in logs → review the pipeline configuration and prevent the issue from happening again.
Example:
Pipeline logs → password or token print ayyinda chudu.
Exposed token → immediately revoke or rotate cheyyi.
CI/CD secret manager → new secret securely store cheyyi.
Pipeline lo echo $PASSWORD remove cheyyi → pipeline rerun chesi secret logs lo kanipinchadam leda verify cheyyi.

---

11. CI/CD pipeline is not triggering after a Git push. What will you check?
Ans: First, I check whether the Git push was successful and whether the pipeline trigger is configured for the correct repository and branch. Then I verify the webhook or trigger configuration.
I also check whether the pipeline has branch filters or path filters that are preventing the trigger. After correcting the trigger configuration, I push a test change and verify that the pipeline starts automatically.
Simple ga gurthupettuko: Git push tarvatha pipeline trigger avvakapothe → first push successful aa chudu → correct repo and branch check cheyyi → webhook or trigger verify cheyyi → branch and path filters check cheyyi → configuration fix chesi test push tho pipeline trigger ayyinda verify cheyyi.
Eng: If the pipeline does not trigger after a Git push → first confirm the push succeeded → check the repository and branch → verify the webhook or trigger → check branch and path filters → fix the configuration and verify the pipeline triggers with a test push.
Example:
git push → push successful ayyinda chudu.
CI/CD settings → repository and branch trigger correct ga unnaya check cheyyi.
Webhook configuration → delivery successful ayyinda verify cheyyi.
Small test change push chesi → pipeline automatically start ayyinda chudu.

---

12. CI/CD pipeline succeeds, but deployment is not updated. What will you check?
Ans: First, I verify whether the pipeline actually deployed the new artifact or Docker image. Then I check the image tag, deployment configuration, and target environment.
For Kubernetes deployments, I check the Deployment image and rollout status to confirm whether the new version was actually deployed. After identifying the issue, I correct the deployment configuration and verify the running version.
Simple ga gurthupettuko: Pipeline success ayina deployment update avvakapothe → first new artifact deploy ayyinda chudu → image tag verify cheyyi → target environment check cheyyi → Kubernetes ayithe Deployment image and rollout check cheyyi → configuration fix chesi running version verify cheyyi.
Eng: If the pipeline succeeds but the deployment is not updated → first verify the new artifact was deployed → check the image tag → verify the target environment → for Kubernetes, check the Deployment image and rollout → fix the configuration and verify the running version.
Example:
docker images → new image tag unda chudu.
kubectl get deployment myapp -n prod -o=jsonpath='{.spec.template.spec.containers[0].image}' → deployed image verify cheyyi.
kubectl rollout status deployment/myapp -n prod → new version rollout ayyinda check cheyyi.
Correct image tag configure chesi → pipeline malli run cheyyi → running Pods new image use chesthunnaya verify cheyyi.

---

13. CI/CD pipeline fails only in production but works in lower environments. What will you check?
Ans: First, I compare the production environment with the lower environments and identify the differences. I check environment variables, secrets, permissions, network access, resources, and external dependencies.
I also verify whether production uses a different configuration, database, registry, or deployment target. After identifying the environment-specific issue, I correct it and rerun the deployment.
Simple ga gurthupettuko: Prod lo matrame fail ayithe → first Prod and lower environment difference chudu → variables and secrets verify cheyyi → permissions and network check cheyyi → resources and external dependencies chudu → Prod-specific configuration fix chesi deployment malli verify cheyyi.
Eng: If the pipeline fails only in production → first compare production with lower environments → verify variables and secrets → check permissions and network → check resources and external dependencies → fix the production-specific configuration and verify the deployment.
Example:
echo $ENVIRONMENT → correct environment value unda chudu.
Environment secrets → required production values available unnaya verify cheyyi.
kubectl config current-context → correct production cluster aa check cheyyi.
Production configuration fix chesi → deployment malli run cheyyi → application healthy ga unda verify cheyyi.

14. CI/CD pipeline is failing because of insufficient permissions. What will you check?
Ans: First, I identify which operation is being denied from the pipeline logs. Then I verify the service account, user, access token, or cloud role used by the pipeline.
I check whether the identity has the required permissions for the target resource and environment. After updating the required permissions according to security policies, I rerun the failed stage and verify the operation.
Simple ga gurthupettuko: Permission valla pipeline fail ayithe → first exact permission error chudu → pipeline identity identify cheyyi → ServiceAccount, token or role check cheyyi → required resource permission verify cheyyi → permission correct chesi failed stage malli run cheyyi.
Eng: If the pipeline fails because of permissions → first check the exact permission error → identify the pipeline identity → verify the ServiceAccount, token, or role → check permissions for the required resource → correct the permission and rerun the failed stage.
Example:
Pipeline logs → permission denied error identify cheyyi.
kubectl auth can-i create deployment -n prod → Kubernetes permission check cheyyi.
Cloud deployment ayithe → pipeline role permissions verify cheyyi.
Required permission add chesi → pipeline stage malli run cheyyi → deployment successful ayyinda verify cheyyi.

---

15. CI/CD deployment succeeded, but the application is down. What will you check?
Ans: First, I check the application health, Pod or container status, and application logs after deployment. Then I verify whether the new version started correctly and whether configuration or environment variables are correct.
I also check Service connectivity, health probes, and resource availability. If the new release caused the issue, I rollback to the previous stable version and investigate the root cause.
Simple ga gurthupettuko: Deployment success ayina app down unte → first application and Pod status chudu → logs check cheyyi → new version and configuration verify cheyyi → Service and health probes check cheyyi → resources verify cheyyi → new release issue ayithe rollback chesi root cause fix cheyyi.
Eng: If the deployment succeeds but the application is down → first check application and Pod status → check logs → verify the new version and configuration → check Service and health probes → verify resources → rollback if the new release caused the issue and fix the root cause.
Example:
kubectl get pods -n prod → Pods Running and Ready ga unnaya chudu.
kubectl logs <pod-name> -n prod → application startup error check cheyyi.
kubectl get svc -n prod → Service correct ga unda verify cheyyi.
New release issue ayithe → previous image/version ki rollback cheyyi → application malli accessible ayyinda verify cheyyi.

---

16. CI/CD pipeline artifact is missing. What will you check?
Ans: First, I check whether the build stage successfully generated the expected artifact. Then I verify the artifact path, filename, and pipeline configuration used to publish it.
I also check whether the artifact was uploaded successfully and whether the next stage is using the correct artifact name and version. After correcting the artifact configuration, I rerun the pipeline and verify that the artifact is available.
Simple ga gurthupettuko: Artifact missing ayithe → first build artifact create ayyinda chudu → correct path and filename check cheyyi → artifact upload successful aa verify cheyyi → next stage correct artifact use chesthunda chudu → configuration fix chesi pipeline malli run cheyyi.
Eng: If the artifact is missing → first verify the build created the artifact → check the path and filename → verify artifact upload → confirm the next stage uses the correct artifact → fix the configuration and rerun the pipeline.
Example:
ls -la target/ → expected artifact create ayyinda chudu.
ls -la build/ → artifact correct path lo unda verify cheyyi.
CI/CD artifact configuration → correct file publish chesthunda check cheyyi.
Pipeline rerun chesi → next stage artifact successfully receive chesthunda verify cheyyi.

---

17. CI/CD pipeline is using an old Docker image. What will you check?
Ans: First, I check the image tag being used by the pipeline and deployment configuration. Then I verify whether the new image was actually built and pushed to the registry.
I also check whether a static tag such as latest is causing confusion or whether the deployment is using an older cached image. After correcting the image tag or pull configuration, I redeploy and verify the running image.
Simple ga gurthupettuko: Old image use avutunte → first pipeline image tag chudu → new image build and push ayyaya verify cheyyi → deployment config check cheyyi → image cache or pull policy verify cheyyi → correct tag use chesi redeploy cheyyi → running image verify cheyyi.
Eng: If the pipeline uses an old Docker image → first check the image tag → verify the new image was built and pushed → check deployment configuration → verify image caching or pull policy → redeploy with the correct tag → verify the running image.
Example:
docker images → new image tag unda chudu.
docker push myregistry/myapp:v2 → new image registry ki push ayyinda verify cheyyi.
kubectl get deployment myapp -n prod -o=jsonpath='{.spec.template.spec.containers[0].image}' → deployment ye image use chesthundo chudu.
Correct image tag configure chesi → deployment malli run cheyyi → new image Pod lo running unda verify cheyyi.

18. CI/CD pipeline rollback is required after a failed deployment. What will you do?
Ans: First, I confirm that the latest release caused the production issue and identify the previous stable version. Then I verify the available image or artifact version that was working correctly.
I rollback to the previous stable version using the deployment mechanism and monitor the application. After rollback, I verify application health, logs, and traffic before investigating the failed release.
Simple ga gurthupettuko: Failed deployment tarvatha rollback kavali ante → first latest release problem aa confirm cheyyi → previous stable version identify cheyyi → stable image or artifact verify cheyyi → rollback cheyyi → application health and traffic check cheyyi → failed release root cause investigate cheyyi.
Eng: If rollback is required after a failed deployment → first confirm the latest release caused the issue → identify the previous stable version → verify the stable image or artifact → rollback → check application health and traffic → investigate the failed release.
Example:
kubectl rollout history deployment/myapp -n prod → previous stable revision chudu.
kubectl rollout undo deployment/myapp -n prod → previous version ki rollback cheyyi.
kubectl rollout status deployment/myapp -n prod → rollback complete ayyinda verify cheyyi.
Application test chesi → traffic and logs normal ga unnaya check cheyyi.

---

19. CI/CD pipeline is successful, but notifications are not being received. What will you check?
Ans: First, I check whether the notification stage actually executed successfully and review its logs. Then I verify the notification channel configuration, webhook, email settings, or integration credentials.
I also check whether the notification is being filtered based on pipeline status or branch conditions. After correcting the integration or configuration, I run a test pipeline and verify that the notification is received.
Simple ga gurthupettuko: Pipeline success ayina notification raakapothe → first notification stage run ayyinda chudu → logs check cheyyi → webhook or email configuration verify cheyyi → credentials check cheyyi → branch or status filters chudu → issue fix chesi test notification verify cheyyi.
Eng: If notifications are not received even though the pipeline succeeds → first check the notification stage → review logs → verify webhook or email configuration → check credentials → verify branch or status filters → fix the issue and test the notification.
Example:
Pipeline logs → notification stage successful ayyinda chudu.
Webhook configuration → correct URL and credentials unnaya verify cheyyi.
Notification filters → successful pipeline ki notification enabled unda check cheyyi.
Test pipeline run chesi → notification receive ayyinda verify cheyyi.

---

20. How will you troubleshoot a complete CI/CD production failure?
Ans: First, I check the pipeline execution, recent code changes, failed stage, and deployment logs to identify where the production issue started. Then I verify the build artifact or Docker image that was deployed.
I also check the production application, infrastructure, configuration, secrets, resources, and network connectivity. If the latest release is confirmed as the cause, I rollback to the last stable version and restore the service. After recovery, I identify the root cause and make the required permanent fix.
Simple ga gurthupettuko: Complete CI/CD Prod failure vaste → first pipeline and failed stage chudu → recent changes check cheyyi → deployed artifact or image verify cheyyi → application, configuration, secrets and resources check cheyyi → network verify cheyyi → latest release issue ayithe rollback cheyyi → service restore chesi root cause fix cheyyi.
Eng: If there is a complete CI/CD production failure → first check the pipeline and failed stage → review recent changes → verify the deployed artifact or image → check application, configuration, secrets, and resources → verify network → rollback if the latest release caused the issue → restore the service and fix the root cause.
Example:
Pipeline logs → first failure ekkada start ayyindo chudu.
kubectl get pods -n prod → production application health check cheyyi.
kubectl logs <pod-name> -n prod → runtime error verify cheyyi.
kubectl rollout history deployment/myapp -n prod → previous stable version identify cheyyi.
kubectl rollout undo deployment/myapp -n prod → stable version ki rollback cheyyi.
kubectl rollout status deployment/myapp -n prod → service recover ayyinda verify cheyyi.
Root cause identify chesi → permanent fix apply cheyyi → pipeline malli run chesi → production deployment successful ga unda verify cheyyi.